Memoria del proceso

Entorno LAMP con Docker: nginx + PHP-FPM 8.3 + MySQL

Alejandro Martínez Sillero

Pasos a seguir (resumen)

1. Crear el repositorio en GitHub, clonarlo y crear las carpetas php, nginx y src. **2.** Escribir php/Dockerfile: imagen php:8.3-fpm e instalar pdo_mysql.
3. Escribir nginx/default.conf con fastcgi_pass php:9000.
4. Escribir docker-compose.yml con los servicios web (8080:80), php y db.
5. Crear src/index.php que se conecte a MySQL con PDO (host = db).
6. Levantar todo: docker compose up -d --build.
7. Comprobar en http://localhost:8080 y hacer la captura de pantalla.
8. Escribir el README.md (que hace, como levantarlo, captura).
9. Hacer commits pequeños en cada paso y subir con git push.

1. Objetivo del proyecto

En este proyecto he montado un servidor web para PHP usando Docker en lugar de instalar XAMPP. En vez de tener Apache, PHP y MySQL mezclados en mi ordenador, cada pieza vive en su propio contenedor: nginx recibe las peticiones, PHP-FPM ejecuta el código y MySQL guarda los datos.

La ventaja es que cada contenedor está aislado y es desechable: si algo se rompe, lo borro y lo vuelvo a crear con un solo comando. Además, es la forma de trabajar que se usa en entornos profesionales, y cualquiera puede levantar mi proyecto igual que yo.

2. Estructura del repositorio

Primero creé el repositorio en GitHub, lo cloné en mi ordenador y organicé los archivos en carpetas, una por cada servicio:

```
proyecto/
  docker-compose.yml
  README.md
  nginx/
  default.conf
  php/
  Dockerfile
  src/
  index.php
```

3. Contenedor php (php-fpm 8.3)

Para PHP partí de la imagen oficial php:8.3-fpm. FPM es el proceso que ejecuta el código PHP y espera peticiones en el puerto 9000 de la red interna de Docker.

Necesité un Dockerfile propio porque la imagen oficial no trae la extensión pdo_mysql, que hace falta para conectar con la base de datos. Por eso, en php/Dockerfile añadí una línea con docker-php-ext-install pdo pdo_mysql.

4. Contenedor web (nginx)

Usé la imagen oficial de nginx y le monté mi propio archivo de configuración, nginx/default.conf. En él le indico que escuche en el puerto 80 del contenedor, que la carpeta de los archivos es /var/www/html y que index.php es el archivo por defecto.

La parte clave es el bloque location para los archivos .php: con fastcgi_pass php:9000 nginx reenvía esas peticiones al contenedor php (Docker resuelve el nombre php como su dirección). En el compose publiqué el puerto como 8080:80, así que desde el navegador entro por http://localhost:8080.

5. Contenedor db (MySQL)

Para la base de datos usé la imagen oficial mysql:8.0. Se configura con variables de entorno: la contraseña de root, el nombre de la base de datos, un usuario y su contraseña.

Añadí un volumen (db_data) para que los datos se conserven aunque borre y recree el contenedor. Las contraseñas del ejemplo son solo de prueba; en un proyecto real irían en un archivo .env que no se sube a GitHub.

6. docker-compose.yml

En el docker-compose.yml defino los tres servicios (web, php y db) en un único archivo. Compose crea automáticamente una red interna donde todos se ven por su nombre de servicio, por eso nginx puede llamar a php y PHP puede llamar a db.

Con volumes comparto la carpeta ./src con nginx y con php, de modo que ambos ven el mismo código y los cambios se reflejan al instante. Con depends_on indico el orden de arranque: primero db, luego php y por último web.

7. Conexión PHP con PDO

En src/index.php creo un objeto PDO con el DSN mysql:host=db;dbname=app, usuario y contraseña. Lo importante es que el host es db (el nombre del servicio) y no localhost, porque localhost dentro del contenedor php sería el propio contenedor.

Envolví la conexión en un try/catch: si funciona, muestro "Conexión a MySQL correcta", y si falla, muestro el mensaje de error. Así compruebo a simple vista que todo está bien conectado.

8. Problemas encontrados y soluciones

502 Bad Gateway: nginx no llegaba a PHP. Revisé que en default.conf pusiera fastcgi_pass php:9000 y que el contenedor php estuviera en marcha con docker compose ps.

Connection refused / could not find driver: en el primer caso el host debía ser db y no localhost; en el segundo faltaba instalar pdo_mysql en el Dockerfile y reconstruir con --build.

Puerto 8080 ocupado: otro programa lo usaba. Lo solucioné parándolo o cambiando el mapeo a otro puerto, por ejemplo 8081:80.

MySQL no está listo al arrancar: tarda unos segundos en iniciarse; si PHP falla al principio, basta con esperar y recargar la página.

9. Conclusiones

He aprendido a separar una aplicación web en servicios independientes y a conectarlos con Docker Compose. Frente a XAMPP, este modelo es más limpio, reproducible y fácil de compartir: cualquiera clona el repositorio, ejecuta docker compose up -d y tiene el mismo entorno que yo.

También he visto lo útil que es ir haciendo commits pequeños en cada paso, porque el historial refleja cómo fui construyendo el proyecto.

10. Historial de commits

1	Initial commit	README vacio y .gitignore
2	Add php Dockerfile	php/Dockerfile con php-fpm 8.3 y pdo_mysql
3	Add nginx config	nginx/default.conf
4	Add docker-compose with web and php	Servicios web y php funcionando
5	Add index.php	src/index.php basico (phpinfo o Hola)
6	Add mysql service	Servicio db con volumen y variables
7	Connect PHP to MySQL with PDO	Conexion y consulta de prueba
8	Add README with screenshot	Documentacion y captura final

11. Captura de pantalla

Pega aqui la captura de http://localhost:8080 mostrando la conexion a MySQL correcta

